MAnager for SoC Test

User Manual

Revision History

25/05/2023: Created

Index

1 Introduction and Principles

The aim of the MAST tool is to provide a flexible and optimized environment for executing PDL-1 based algorithms around the IEEE 1687-2014 standard, when traditional IJTAG tools are focused on pattern generation rather than on their application. As such, it does not strive to provide a complete implementation of the standard, but rather showcase its most disruptive features in terms of Algorithmics, Dynamic Reconfiguration and Interactive Execution. However, the MAST Kernel is by nature modular and extensible, so an interested user will easily be able to add the desired features. MAST main principle is "if you designed it, you must be able to use it". As such, the main aims of the software are to support:

- **ANY PDL-1 algorithm**, regardless of its complexity (or simplicity)
- **ANY Topology,** being it SIB-based or using custom hierarchy-enabling elements
- **ANY Interface,** being it JTAG or other, by implementing and prototyping P1687.1 concepts
- **TRANSPARENT RETARGETING**: any PDL-1 algorithms must be immediately retargetable through any topology and/or any interface without any modification or user intervention;
- **MASSIVE CONCURRENCY**: MAST must support concurrent execution of PDL-1 Algorithms for next-generation SoCs, which will have hundreds of instruments to handle
- **INTERACTIVE EXECUTION**: traditional testing is based on offline vector generation. MAST allows PDL-1 Algorithms to be executed dynamically against real targets
- **FULL PORTABILITY**: MAST relies only on standard libraries, so it can be executed on any target supporting multithreading (Windows and Linux, both on desktop and embedded targets).
- **BACKWARD COMPATIBILITY**: MAST can be used also for offline retargeting to generate pattern files (SVF, STIL, etc..) which can be used in traditional Test Flows.

This document provides a basic introduction to the main features of MAST for a new user. The justification of its performances can be found in the papers cited in the **References** section. Details of the internal implementation of MAST can be found in the MAST User Manual.

MAST workflow, depicted in Figure 1, is rather different than tradition ATPG-based tools.

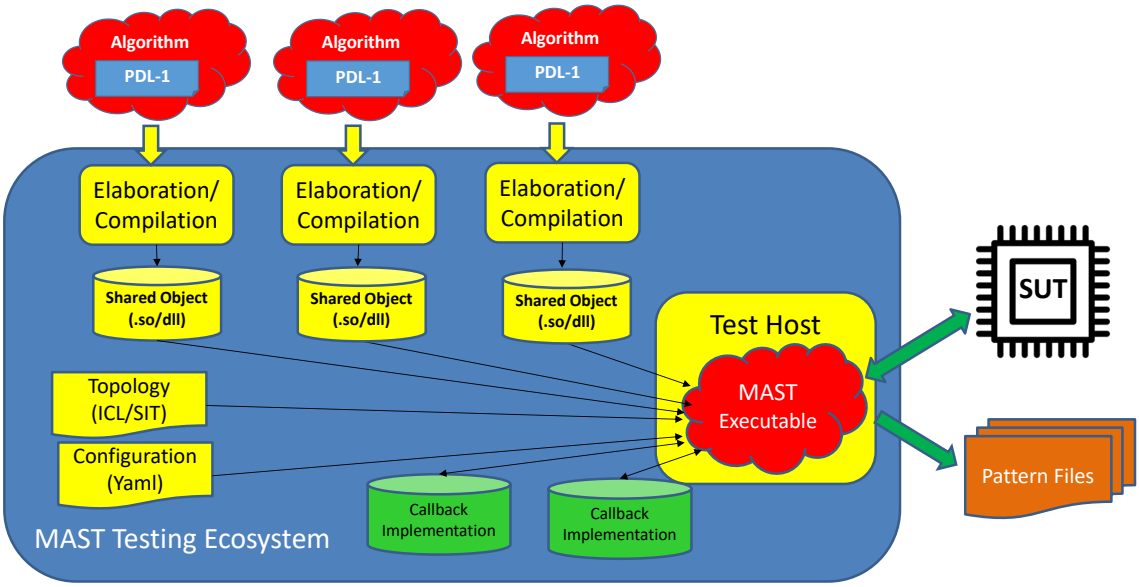

Figure 1 MAST Top-level Execution Flow

The core of the Ecosystem is the MAST kernel, depicted in the right-hand side, running on the Test Host. Upon execution, it loads the Topology for the System Under Test from the ICL and/or SIT[1] files to build its internal Circuit Model. This model is then populated using the Interface and Translator extensions provided as Shared Object Callbacks, following P1687.1 concept[2]. Next, the pre-compiled PDL-1 algorithms are loaded and associated with the corresponding Register. Last, MAST launches the execution of all loaded PDL-1 Algorithms. The principle of MAST is concurrency: PDL routines are associated with the Register they refer too, and are all executed in parallel. Instead of creating a merged top-level PDL, MAST directly generates the top-level operations using P1687.1 concepts. MAST can either work offline as a traditional retargeter, generating output Pattern Files for later usage, or dynamically interact with the System Under Test providing Vectors and collecting return Data, which is then re-injected into the Circuit Model. Execution stops when all PDL-1 algorithms have exited.

[1] See the dedicated Documentation for information about the “Simplified ICL Tree” (SIT) Format.

[2] See the dedicated Documentation for information about Callbacks and P1687.1

2 Quickstart

MAST is launched from the command line: the only compulsory parameter is the Yaml configuration file. To execute the provided examples:

Go to the installation directory:

```
cd cmake_debug/Install/Examples
```

Execute the JTAG example (nb: "Mast" is actually a symlink to the real executable)

```
./Mast -c=Examples.yml -s=./SIT/JTAG.sit
```

By default, MAST generates three output files:

- "Mast.log", containing execution information. Its level of details can be parametrized in the Yaml file;
- "MastModel.txt" : a text-file representation of the internal Circuit Model built from the input topology files;
- "MastModel.gml": a tree representation of the internal Circuit Model built from the input topology files, expressed in GML[3] format. It can be displayed with tools such as the yEd Graph Editor[4]

The configuration file, whose detailed syntax is provided in the Doxygen Documentation, provides all the information needed for execution. Please note that command-line options can be used to override almost all parameter provided in the configuration file.

The main pieces of information are:

- The top-level topology file (ICL or SIT);
- The directory where run-time plugins (i.e. Callback implementations) can be found
- The plugin containing the main PDL-1 procedure, if it is not in the Plugin Directory

Figure 2 depicts the typical MAST Execution setup:

[3] https://en.wikipedia.org/wiki/Graph_Modelling_Language

[4] https://www.yworks.com/products/yed

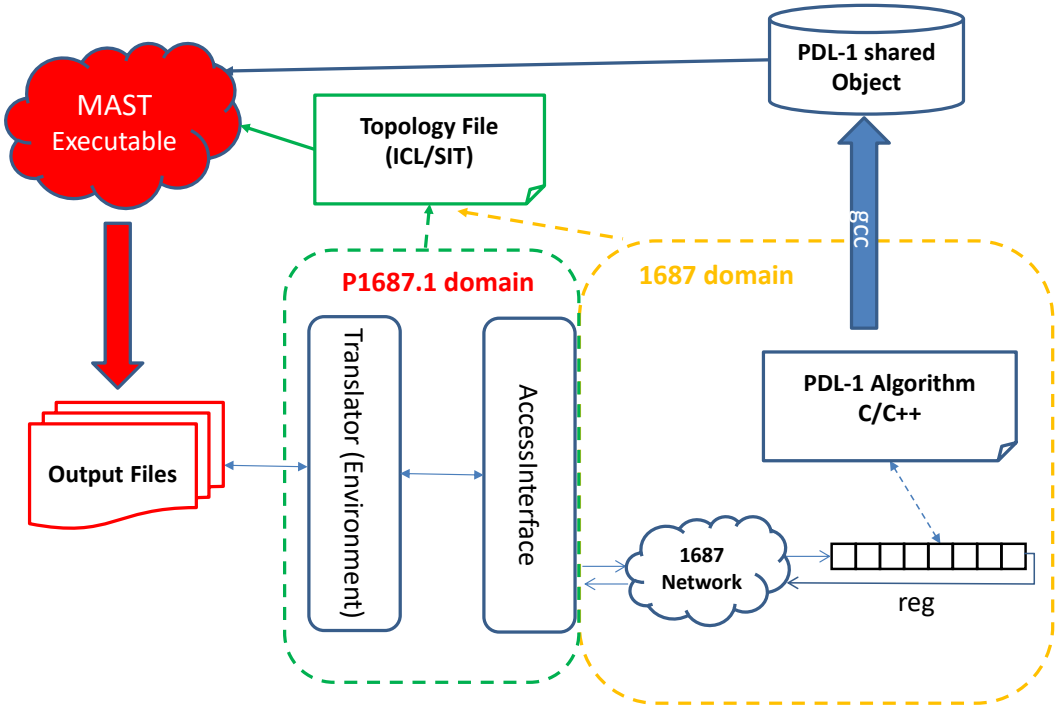

Figure 2 Mast Execution Setup

The starting point is the right-hand side: a 1687 domain where an Network is defined, with one or more PDL-1 algorithms (written in C/C++) associated with one or more registers. This network can be described either in ICL or in SIT. The connection to the outside, in the center, is expressed in P1687.1 terms[5]: an AccessInterface formats 1687 primitives into the Interface once. For instance, in the case of a TAP it traduces Capture-Shift-Update (CSU) operations into SIR and SDR operation. The last step is the connection to the outside world, i.e. the Execution Environment. MAST models is as P1687.1 Interface, whose role is either to generate the offline output files for later application (ex: SVF) or directly interact with the SUT. MAST implements natively 3 such Environmentss:

- "Emulation": this allows the Emulation of a 1687 execution to generate pattern files, logged into a "Log.txt" file. Interactive behavior is emulated by making a loopback: data sent "to the SUT" is used also as data received "from the SUT".

[5] As P1687.1 has not been finalized yet, the presented nomenclature is proper only to MAST. It will be adapted once the standard is ratified.

- "Simulation": this allows a co-simulation with an RTL simulator such as Modelsim or VCD. "To SUT" data is sent to the simulator, and "from SUT" data is collected from it.
- "FTDI": this allows interactive execution against a real SUT using and FTDI USB dongle[6].

3 Examples

In this section, we will provide a detailed explanation of the provided examples.

3.1 JTAG.sit

To start, we will consider the simplest possible example: a 1687 register "reg" connected to an 1149.1 TAP and a wrapped-PDL function "Algo_increment" acting on the register, as depicted in Figure 3

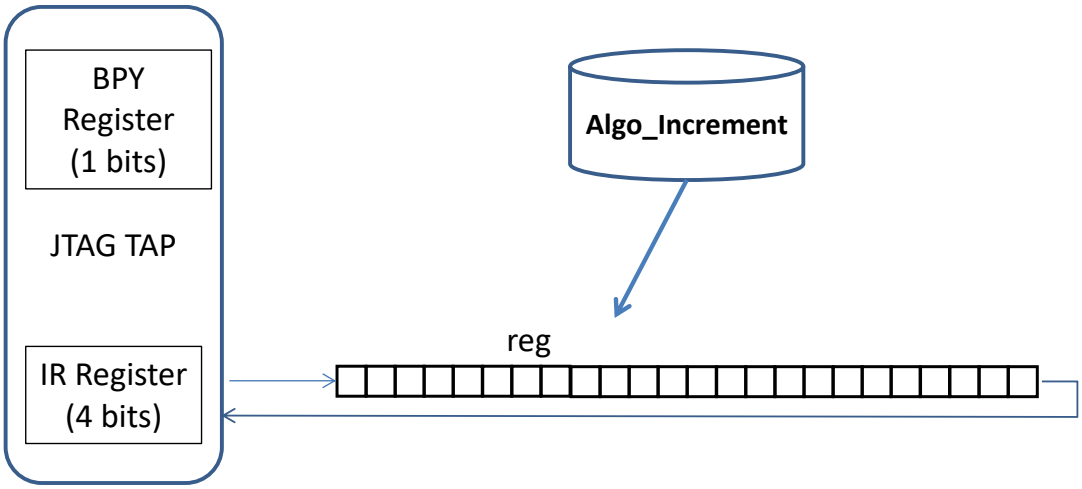

Figure 3 Example setup for JTAG.sit

The command line is :

```
./Mast -c=Examples.yml -s=./SIT/JTAG.sit
```

The Wrapped PDL-1 implements a simple counter loop: at each cycle an "iGet" reads the value of the register to print in on screen, while an "iWrite" increments it.

```
void Algo_Increment ()
```

[6] At this moment in time, FTDI only works on Linux for JTAG targets.

```
{

  auto     registerPath = "reg";
  auto     loopCount    = 5u;
  auto     i    = 0u;
  uint16_t initialValue = 1u;
  uint16_t curValue;

 curValue=iGetRefresh<uint16_t>(registerPath);
  std::cout << "\nINCREMENT Running " << loopCount << " iWrites on
register " <<registerPath << "\n";
  iNote(iNoteType::Comment,"Let's go!");
  while (i++<loopCount)
  {
    iWrite(registerPath, initialValue);
    iApply();
    curValue=iGet<uint16_t>(registerPath);
   std::cout << "\nINCREMENT Cycle "<< i
              << ": Wrote " << initialValue ;
   std::cout << "\nINCREMENT       "<< i
              << ": Read " <<std::hex<< curValue <<"\n" ;

    ++initialValue;
  }
   std::cout << "\n" ;
}
```

Several points are worth mentioning:

- Register identification in iWrite is done through its name, as mandated by 1687;
- The value passed to iWrite is directly taken from the native “intialValue” variable;
- The value obtained from iGet is directly stored in the native C++ variable curValue;
- The whole wrapper is written in standard C++ and can exploit all its features;

In the same file, the function is associated with a symbol for reference in the SIT file:

```
bool RegisterAlgorithms ()
{
    [...]
  repo.RegisterAlgorithm("Console_Incr", Algo_Increment);
  [...]

  return true;
}
```

The reason for this step is name mangling: the compiler changes the names of functions for its internal processing, so it is not possible to directly reference them (in truth, this is theoretically

possible through compiler reverse-engineering, but it is highly discouraged). “repo” is actually the instance of a Factory MAST uses for storing the “function <-> symbol” association, and it can be used thanks to the "PDL_AlgorithmsRepository.hpp" API. Details of this mechanism can be found in either the Doxygen Documentation or the MAST Developer Manual.

The SIT file for the System Under Test is extremely simple:

```
TRANSLATOR top Emulation
 (
 JTAG_TAP TAP 4 1
 PDL Console_Incr
  (
  REGISTER reg 12 Bypass: "0x000"
  )
 )
```

In details:

- “reg” is only described by its size (12 bits) and its bypass value (0x000)
- The TAP is declared by giving the size of its IR register (4) and the number of chains (1). As no IR coding is given, it is automatically set.
- The Environment protocol is set to “Emulation”: this means that execution will use an emulated target implementing a loopback, with TDI hard-wired to TDO. This mode is extremely useful for debugging as it is possible to have predictable results without needing a full simulation backend.
- The function Algo_Increment is associated via its symbol “Console_Incr” to the top-level TAP.

This is the refence output

```
(1)    INCREMENT Running 5 iWrites on register reg
(2)
(3)    INCREMENT Cycle 1: Wrote 1
(4)    INCREMENT       1: Read 1
(5)
(6)    INCREMENT Cycle 2: Wrote 2
(7)    INCREMENT       2: Read 2
(8)
(9)    INCREMENT Cycle 3: Wrote 3
(10)   INCREMENT       3: Read 3
```

```
(11)
(12)   INCREMENT Cycle 4: Wrote 4
(13)   INCREMENT       4: Read 4
(14)
(15)   INCREMENT Cycle 5: Wrote 5
(16)   INCREMENT       5: Read 5
```

The computed TAP-Level operations can be found in the “Emulation.log” file:

```
>More Emulation.log
      SIR 4 TDI(01);
      SDR 12 TDI(0000);
      SDR 12 TDI(0001);
      SDR 12 TDI(0002);
      SDR 12 TDI(0003);
      SDR 12 TDI(0004);
      SDR 12 TDI(0005);
```

The same information can also be extracted from the overall log file:

```
 > grep TD. Mast.log
[Emulation_TranslationProtocol.cpp][TransformationCallback]: SIR 4 TDI(01);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0000);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0001);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0002);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0003);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0004);
[Emulation_TranslationProtocol.cpp][TransformationCallback : SDR 12 TDI(0005);
```

The Circuit Model generated by MAST is dumped in MastModel.txt. It is a one-to-one representation of the SIT input:

```
[Access_T](0)  "top"
 [Access_I](1)  "TAP"
  [Register](2)  "TAP_IR", length: 4, Hold value: true, bypass: 1111
  [Linker](3)    "TAP_DR_Mux"
   :Selector:(2)  "TAP_IR"
   [Register](4)  "TAP_BPY", length: 1, bypass: 1
   [Register](5)  "reg", length: 12, bypass: 0000_0000:0000
```

yEd can be used to display the MastModel.txt, as in Figure 4

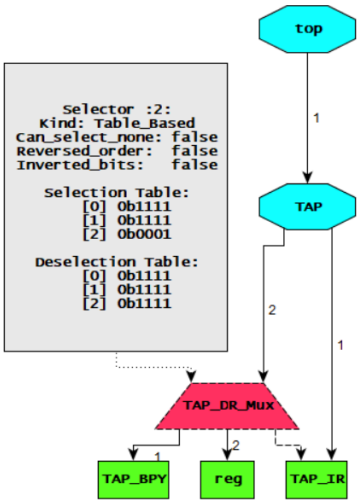

Figure 4 yEd display of MastModel.gml (hierarchical layout)

3.2 ICL JTAG

The same example is available with an ICL description in the ICL/JTAG directory. To use the ICL description, you can override the MAST command line to point to the "lst" file[7] which contains the list of input ICL and BDSL files :

```
./Mast -c=Examples.yml --icl_list=ICL/JTAG/JTAG.lst
```

Please not that :

- the link between ICL and PDL is done in line 6 of the Tap_and_SReg.icl file as a custom attribute
- as P1687.1 still does not specify ICL construct, the connection to the Environment is done with a custom attribute in line 9 of the same file

3.3 PDL Reuse with different topologies

The Examples/SIT folder contains a set of examples topologies to demonstrate the reuse of PDL: all networks contain a "reg" register to which the "Console_Incr" PDL-1 algorithm is associated. To run them, simply change the SIT file reference in the MAST command line, as for instance:

[7] The IEEE 1687-2014 Standard does not specify the handling of multiple files, so MAST simply uses a list of the input files.

```
./Mast -c=Examples.yml -s=./SIT/SIB_Tutorial.sit
```

For each example you can check:

- The Console output should be the same, to prove portability
- MastModel.txt and MastModel.yml to see the topology
- Emulation.log to see the generated output.

Please refer to the README for a detail of the provide examples.

3.4 ICL Leds

The “ICL_Leds” examples want to showcase a more complex usage of ICL . It can be executed as:

```
./Mast -c= ICL_Leds.yml --icl_list=ICL/ICL_Leds/ICL_Leds.lst
```

The example mimics the test of an Leds, tested through a read-back of written values. The Test is encapsulated into a Wrapped Instrument that is replicated three times in a daisy-chain structure.

Please not that:

- The PDL does not have any console output, as most Test programs do
- This example showcases MAST’s capability of concurrent execution: the PDL is instantiated three times and executed in true dynamic parallelism, with no static merging.
- The PDL-1 makes extensive usage of C++11 features to improve performances and readability of the test function.
- You can observe MAST’s execution in the Mast.log file. You can for instance extract the generated SVF with this command line: `grep S.R Mast.log`
- The topology/algorithm coupling is done in ICL: it is line 9 of Instrument

3.5 Concurrency

The ./SIT/Concurrent repository contains a set of examples showcasing MAST capability to support extreme concurrent execution of PDL-1 algorithms. The PDL-1 routine is defined in ./PDL/Random.cpp and performs “loopCount” writes of random data to an 8-bit register. The provided files are:

- ./Concurrent/Random.sit : Example with one-register
- ./Concurrent/Module_8.sit : Wrapping of one-register instance
- ./Concurrent/Replica_100x8.sit: Wrapping of 100 daisy-chain instances of the Wrapped Register, each one executing one instance of the PDL-1 function
- ./Concurrent/Random_horizontal.sit : 4 daisy-chain instances of the Wrapped Register, each one executing one instance of the PDL-1 function
- ./Concurrent/Random_horizontal_stress.sit: 1000 daisy-chain instances of the Wrapped Register, each one executing one instance of the PDL-1 function. A total of 1000 PDL-1 concurrent functions
- ./Concurrent/Random_hierarchical.sit : 1 of the Wrapped Register behind 1 SIB, executing one instance of the PDL-1 function
- ./Concurrent/Random_hierarchical_deep.sit: 1 of the Wrapped Register behind 10 nested SIBs, executing one instance of the PDL-1 function
- ./Concurrent/Random_hierarchical_stress.sit : 90 Level of nested SIBs, with of 100 daisy-chain instances of the Wrapped Register at each level, each one executing one instance of the PDL-1 function. A total of 9000 PDL-1 concurrent functions

  The examples can be executed as usual way from the Install directory

  ```
  ./Mast -c=Concurrent.yml
  ```

  or

  ```
  ./Mast -c=Concurrent.yml -s=./Concurrent/ Random_hierarchical_stres.sit
  ```

3.6 RTL Co-simulation

All examples can be co-simulated against an RTL model by using the dedicated Environment Translator “Simulation”. For instance for JTAG.sit:

```
TRANSLATOR top Simulation
 (
 JTAG_TAP TAP 4 1
 PDL Console_Incr
  (
  REGISTER reg 12 Bypass: "0x000"
  )
 )
```

The “Simulation” Translation generates SVF transactions like in the precedent case, but instead of simply logging them they are sent to an RTL simulator, for instance Modelsim, through two exchange files.

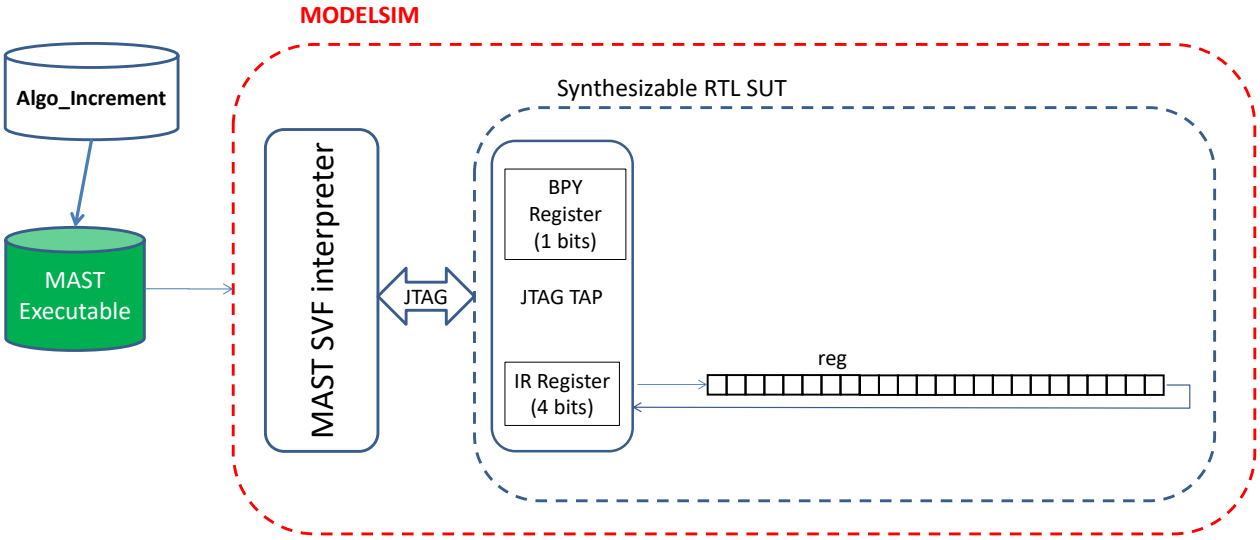

Figure 5 Co-Simulation Setup

Figure 5 shows the complete setup: the RTL model of the target system is simulated in Modelsim, and its TAP is driven by an SVF interpreter. This interpreter, written in standard VHDL, is provided with MAST and is able to receive commands from the external MAST executable, apply vectors to the SUT and send back output vectors. The “Algo_increment” function is therefore able to communicate directly with the RTL implementation of the target system. Please note that you must simulate the same system described in the SIT file by correctly select it in the “MAST_config.vhdl” file.

For details on how to perform co-simulation please refer to the README in the ./vhd directory.

3.6 Cryptographic examples

The repository contains an example of the usage of the **Trivium stream** cypher to encrypt the data on the scan chain [5]: by modeling the encryption/decryption module as a P1687.1 Translator, MAST is able to support security as part of the Standard flow, allowing plug-and-play usage [5]. Please note that the reference Trivium implementation is a Python file, so MAST uses the "pyhelper.hpp" and "<Python.h>" libraries to call Python scripts from C++ modules. Although powerful, the library does not provide much error checking: almost all problems in the Python installation and configuration will cause segmentation faults: the repository gives a working example, but it you're your responsibility to adapt them to your setup.

First, configure your environment to look for the Python script in the current directory

```
source ./setupTrivium.sh
```

You can now run the example:

```
./Mast -c=Examples.yml -s=./SIT/Trivium.sit
```

Upon execution, the Trivium.sit example provides the same console output. However, an analysis of "Emulation.log" will show how the data exchanged on the scan chain is actually ciphered.

A cosimulation is also possible:

- Go to the RTL directory

  ```
  cd ./RTL/VHDL
  ```

- Compile the RTL files as usual:

  ```
  Source ./compile_VHDL
  ```

- Open Modelsim and prepare simulation:

  ```
  do Trivium.do
  ```

The script will do three things:

1- Load the SVF_simulation_top_encryption, which instantiated a Trivium stream cypher between the Master and Slave TAPs

2- Prepare the Wave window for easier debug

3- Run the testbench to initialize the cypher

You can now run Mast using the usual cosimulation setup:

```
./Mast -c=Cosim.yml -s=./SIT/Trivium.sit
```

Don't forget to launch the Modelsim simulation (ex: "run -all") until Mast execution stops.

In the Wave window, you can notable observe in the last two lines the INPUT values which are read by iGet and the OUTPUT values which are written by iWrite.

NB: the provided SIT file supposes the VHDL to be configured for the "tutorial_1" example. All provided VHDL examples can be executed using Trivivium by loading "SVF_Simulation_top_encryption" in VSIM and modifying the SIT file appropriately.

4 Software and Repository organization

The MAST software follows a modular organization, as depicted in Figure 6

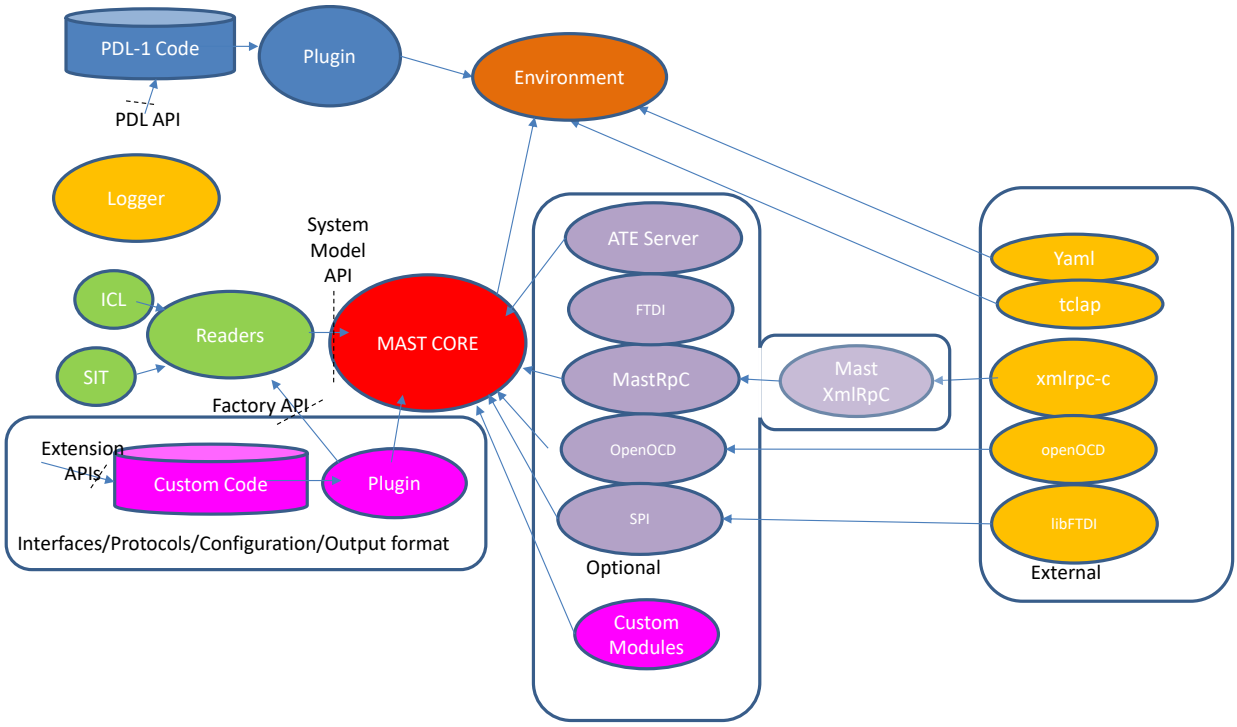

Figure 6 MAST Software organization

The main element is the MAST Core, or Kernel, depicted in Red in the middle of the picture. It corresponds to the main executable and contains all the Manager and the other central functionalities, as explained in the "MAST Developer Manual". It is complemented by several additional modules, implemented as Dynamic Libraries which are loaded at runtime:

- "Environment" takes care of parsing the command-line, prepare and launch the Core and other housekeeping functions;
- The "Readers" are used to build the internal System Model based on the input Domain-Specific-Language. At this moment in time, two Readers are implemented: ICL and SIT;
- "PDL-1 Code" can be compiled and built into a dynamic library following the instructions provided in this document. At start-up, "Environment" loads all Dynamic

Libraries available in the directory pointed by the configuration file (please refer to the Doxygen documentation);

- “Logger” provides asynchronous logging and debug capabilities through the system;
- “Optional” modules implement custom functionalities that are not necessary to the base MAST kernel, but which are needed for specific examples or applications. Their inclusion in the build process is parametrized in the “UserOptions.cmake” file;
- “External” regroups the external libraries needed by internal or optional MAST modules. They are handled separately to avoid dependency with their own repositories. Their inclusion in the build process is either parametrized in the “UserOptions.cmake” file or explained in their READMEs;

5 Custom User Extensions

5.1 User PDL functions

It is extremely easy to write your own PDL-1 routines by using the provided C++ files as an example. The main elements are:

- Include the base libraries. The most important 4 are :
  - "CPP_API.hpp" : API for PDL commands
  - "BinaryVector.hpp" : C++ types for handling binary vectors
  - "PDL_AlgorithmsRepository.hpp" : API to register PDL functions for SIT/ICL reference
  - "g3log/g3log.hpp" : logging facility to “Mast.log”
- Write your C++ function following the prototype:

```
void <function_name> ()
```

- Register the symbolic name to avoid name mangling following the templace:

```
bool RegisterAlgorithms ()
{
  // ---------------- Get an handle on PDL algorithm repository
  //
  auto& repo = PDL_AlgorithmsRepository::Instance();

  // ---------------- Do register algorithm(s) with a name
  //
  repo.RegisterAlgorithm("<ICL_Symbol>", []() { <function_name>();
});

  return true;
}
```

- Use the ./Examples/Cmakefiles.txt as an example to compile you file
- Verify that your YAML configuration file points to the generated shared library

5.2 Custom P1687.1-like Translators

A previous stated, MAST defines Translators exclusively in software following a standard API specification, following the principles that are under discussion in the P1687.1 Working Group [3]. The Standard is not finalized yet, so this feature must not be considered in any way a reference implementation of P1687.1: it is just a custom implementation of what the standard might look like.

A "Empty" translator can be found in `./Optional_Libs/Empty_TranslatorProtocol/` : it can be used as a basis for implementing a custom Translator following the principles of [4]. The `./Optional_Libs/Trivium` directory is an implementation of a Custom Translator used to model a Secure Scan Encryption scheme [4].

A dedicated White Paper is under development to explain the process in details.

5.3 Custom "Mux Callbacks"

In the IEEE 1687-2014 standard, it is possible to model dynamic elements outside of the IJTAG control as "Callback Registers": to access these elements, the retargeter will execute a specific set of PDL operations instead of resolving their connectivity as usual. MAST extends this concept to Muxes : it is possible to define C++ "PathSelector" functions that allows its custom configuration to be included into the MAST retargeter.

A "Dummy" Path Selector translator can be found in `./Optional_Libs/Dummy_PathSelector/` : it can be used as a basis for implementing a custom PathSelector following the principles of [4]. The `./Optional_Libs/SSAK` directory is an implementation of a Custom PathSelector used to model a Secure Authentication-based Mux [4].

A dedicated White Paper is under development to explain the process in details.

6 Cross compilation

The repository provides Toolchain files for cross-compiling to both ARM and RISCV using Cmake. The Makefile provide “make arm” and “make riscV32” recipes to call Cmake with the right parameters. Please note that :

- Toolchain files are given as a reference: it is your responsibility to adapt them to your installation.
- Some Optional modules are not fully debugged for Cross-compilation. You should set all includes in UserOptions.cmake to OFF
- “make pack_arm” and “make pack_riscV” provide ready-to-export tarballs using Cpack.

7 Annex

7.1 Legacy Modules

In the Repository, some legacy modules are still available for usage for simplified examples, ICL and for the Unit Tests. These are usually custom solutions developed, prototyped and debugged before the final abstraction was obtained. Example are the “Loopback” or “SVF_Emulation”. Please refer to the Doxygen Documentation for detailed information.

References

[1] Portolan M., “Automated Test Flow: the Present and the Future”, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (IEEE-TCAD), December 2019, DOI:10.1109/TCAD.2019.2961328

[2] Portolan Michele, “A novel test generation and application flow for functional access to IEEE 1687 instruments”, Test Symposium (ETS), 2016 21th IEEE European, 23-27 May 2016, **DOI:** 10.1109/ETS.2016.7519302

[3] Portolan Michele, “Accessing 1687 Systems Using Arbitrary Protocols”, Test Conference (ITC), 2016 IEEE International, November 2016

[4] M. Portolan, V. Reynaud, P.Maistri, R. Leveugle, G. Di Natale “Security EDA Extension through P1687.1 and 1687 Callbacks”, , 2021 International Test Conference (ITC21), November 2021, ISBN: 978-1-6654-1695-5, ISSN: 2378-2250

[5] M. Portolan, E. Valea, P. Maistri, G. Di Natale, "Flexible and Portable Management of Secure Scan Implementations Exploiting P1687.1 Extensions", IEEE Design & Test on 30/9.2021, DOI : 10.1109/MDAT.2021.3117875

[6] Portolan M., Reynaud V., Maistri P., Leveugle R., “Dynamic Authentication-Based Secure Access to Test Infrastructure”, 2020 European Test Symposium (ETS 2020), Tallin, ESTONIA, 25 mai au 1 juin 2020